

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA1206	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	U.Prabhavathi	Reg.No.:	192472067
Department:	CSE-AI	Date:	27-08-26

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
 - Maximum interrupt latency < 10 μ s
 - Use vectored interrupts for high-priority devices
 - Use software interrupts for background tasks
-

Code of Conduct:

I **U.Prabhavathi/192472067** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:U.Prabhavathi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. I/O Communication Mechanism for a Real-Time Embedded System

Given Constraints

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Must use vectored interrupt handling
- Choose between memory-mapped I/O and I/O-mapped I/O
- System must provide low interrupt latency

Proposed Solution

For a real-time embedded system, the I/O subsystem should be designed to respond quickly to sensor events while keeping CPU utilization low.

I/O Selection

Memory-Mapped I/O (MMIO) is selected because the registers of the sensors can be mapped into the processor's address space. The CPU can access sensor control and status registers using normal load/store instructions.

Advantages:

- Low access overhead
- Simple programming model
- Fast register access
- Suitable for embedded processors
- Easy integration with DMA

Vectored Interrupt Handling

A vectored interrupt controller is used. Each sensor is assigned a separate interrupt vector.

For example:

Sensor 1 → Vector 1 → ISR 1

Sensor 2 → Vector 2 → ISR 2

Sensor 3 → Vector 3 → ISR 3

Sensor 4 → Vector 4 → ISR 4

When a sensor generates an interrupt, the processor directly jumps to the appropriate ISR instead of checking every sensor.

Reducing CPU Utilization

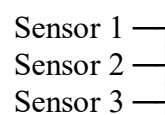
Since sensors interrupt every 1 ms, continuously processing the complete sensor data inside the ISR would increase CPU utilization.

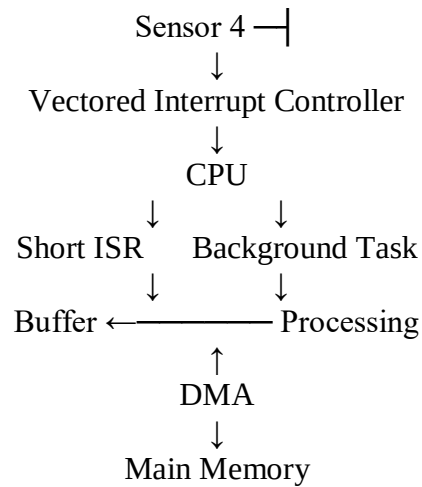
Therefore:

1. ISR identifies the sensor.
2. ISR reads the minimum required status/data.
3. Data is placed into a buffer.
4. ISR exits quickly.
5. A background task processes the data.

DMA can also be used for sensors producing large amounts of data.

Proposed Architecture





Result

The architecture provides:

- Fast interrupt response
- Low ISR execution time
- Reduced CPU utilization
- Efficient sensor communication
- Deterministic real-time behavior

The CPU utilization target of below 15% can be maintained by minimizing ISR work and transferring bulk data using DMA.

2. High-Speed NVMe I/O Subsystem

Given Constraints

- Storage throughput must be greater than 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- DMA must be integrated
- Interrupt notification is required

Proposed Solution

An NVMe SSD connected through PCIe is used as the high-speed storage device.

PCIe provides high-bandwidth communication between the processor/system memory and the NVMe controller.

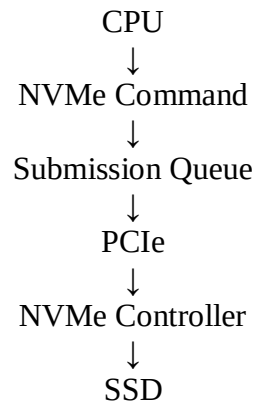
NVMe Communication

NVMe uses queues for communication:

- Submission Queue
- Completion Queue

The CPU places a command in the submission queue.

For example:

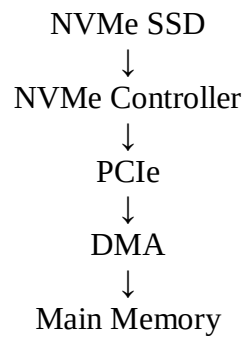


DMA Operation

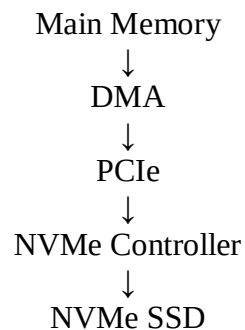
The CPU should not copy every block of data.

Instead, the NVMe controller uses DMA to transfer data directly between the SSD and system memory.

For a read operation:

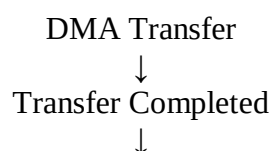


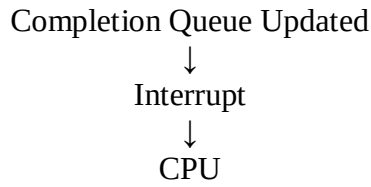
For a write operation:



Interrupt Notification

After completing an operation, the NVMe controller updates the completion queue and generates an interrupt.





The CPU only handles:

- Command preparation
- Queue management
- Interrupt handling
- Completion processing

It does not perform bulk copying.

Result

This architecture provides:

- High storage throughput
- Low CPU overhead
- Efficient DMA transfers
- Fast PCIe communication
- Efficient interrupt notification

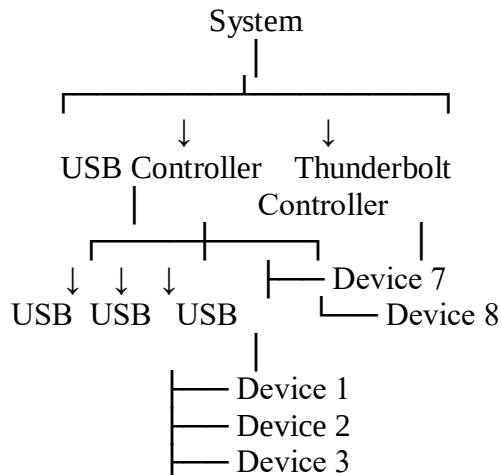
3. Multi-Device Communication Architecture

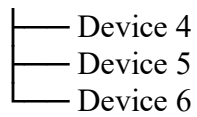
Given Constraints

- 6 USB peripherals
- 2 high-speed Thunderbolt devices
- Avoid interrupt starvation
- Hardware and software interrupts
- Fair bus arbitration

Proposed Architecture

The devices are divided into two groups.





Hardware Interrupts

Hardware interrupts are used for important events such as:

- Data arrival
- Device errors
- Transfer completion
- Thunderbolt high-speed events

Software Interrupts

Software interrupts can be used for:

- Deferred processing
- Background communication
- Protocol processing
- Non-critical tasks

This prevents the hardware ISR from becoming too long.

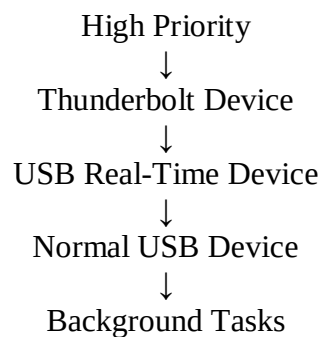
Avoiding Interrupt Starvation

If one device generates a large number of interrupts, it could prevent other devices from being serviced.

To avoid this:

- Assign interrupt priorities.
- Use interrupt moderation/coalescing.
- Keep ISRs short.
- Use deferred processing.
- Give each device service opportunities.

Example:

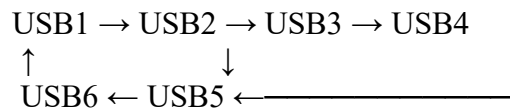


Fair Bus Arbitration

A fair arbitration mechanism ensures that no device permanently occupies the bus.

Round-robin arbitration can be used for normal devices.

Example:



Each device receives an opportunity to access the bus.

For high-speed Thunderbolt devices, bandwidth can be allocated according to system requirements while maintaining fairness.

Result

The system achieves:

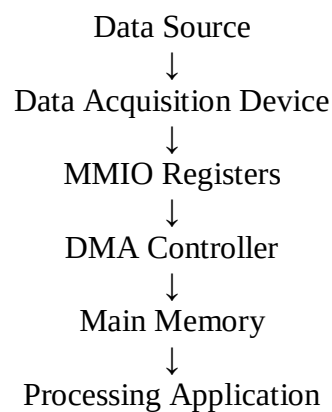
- Fair device access
- No interrupt starvation
- Efficient USB communication
- High-speed Thunderbolt communication
- Proper separation of critical and background tasks

4. Data Acquisition System

Given Constraints

- Continuous data stream of 500 MB/s
- Minimal CPU intervention
- Compare DMA and interrupt-driven I/O
- Device registers must use memory-mapped I/O

Proposed Architecture



Memory-Mapped I/O

The acquisition device's registers are mapped into the CPU's memory address space.

Registers may include:

- Control Register

- Status Register
- Data Register
- DMA Configuration Register
- Interrupt Register

The CPU uses normal memory read/write operations to configure these registers.

Interrupt-Driven I/O

In interrupt-driven I/O, the device generates interrupts when data is available.

At 500 MB/s, generating an interrupt for every small data unit would produce an extremely high interrupt rate.

This causes:

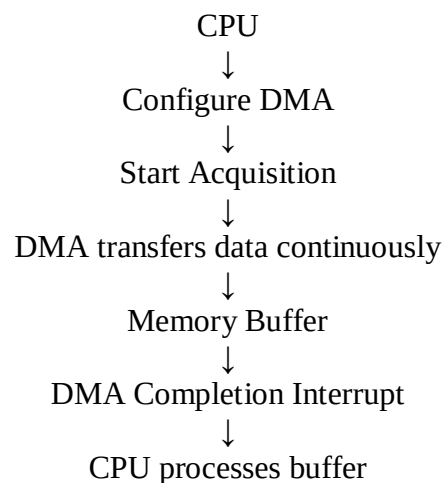
- High CPU utilization
- Increased interrupt overhead
- Reduced processing time
- Possible data loss
- Poor scalability

Therefore, interrupt-driven I/O is not suitable for the complete data stream.

DMA-Based I/O

DMA is much better for this requirement.

The CPU configures the DMA controller once:



Instead of one interrupt per data unit, an interrupt can occur after a **block or buffer** is transferred.

Comparison

Feature	Interrupt-Driven I/O	DMA
CPU involvement	High	Low
Transfer speed	Lower	High

Feature	Interrupt-Driven I/O	DMA
Interrupt frequency	High	Low
CPU overhead	High	Low
500 MB/s stream	Poor choice	Best choice
Continuous transfer	Difficult	Suitable
Buffer-based transfer	Limited	Excellent

Double Buffering

For continuous data acquisition, **double buffering** can be used.

DMA → Buffer A → Processing

↓

DMA → Buffer B → Processing

↓

DMA → Buffer A → Processing

While DMA fills one buffer, the CPU processes the other buffer.

This reduces the possibility of data loss.

Result

DMA with MMIO control registers provides:

- Continuous 500 MB/s transfer
- Minimal CPU intervention
- Efficient buffer management
- Reduced interrupt overhead
- Reliable data acquisition

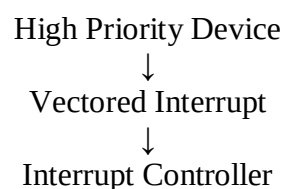
5. Interrupt Handling Mechanism

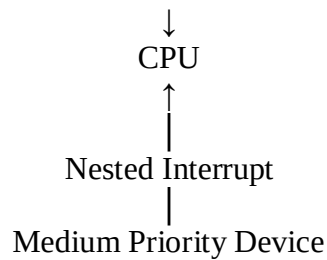
Given Constraints

- Nested interrupts required
- Maximum interrupt latency <10 μ s
- Vectored interrupts for high-priority devices
- Software interrupts for background tasks

Proposed Architecture

A **priority-based interrupt controller** is used.





Interrupt Priority

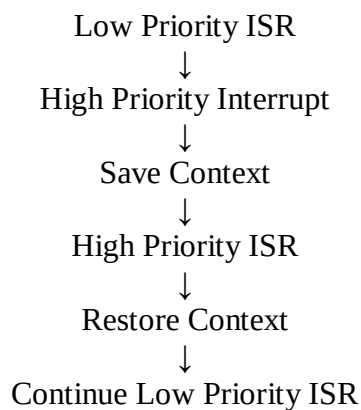
Interrupts can be divided into levels:

Priority	Interrupt	Handling
1	Emergency/critical device	Immediate hardware ISR
2	High-speed device	Vectored ISR
3	Normal device	Hardware ISR
4	Background processing	Software interrupt

Nested Interrupts

Suppose the CPU is processing a low-priority interrupt and a high-priority device generates an interrupt.

The high-priority interrupt should immediately interrupt the lower-priority ISR.



Maintaining <10 μs Latency

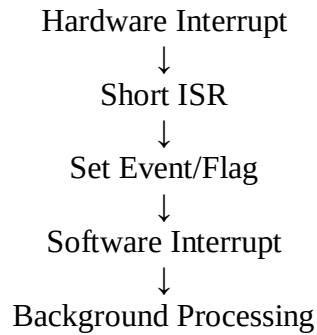
To meet the latency requirement:

1. Use vectored interrupts.
2. Keep ISRs very short.
3. Avoid lengthy calculations inside ISRs.
4. Use priority-based interrupt handling.
5. Use fast context saving/restoring.
6. Use DMA for large data transfers.
7. Defer non-critical processing.

Software Interrupts

Software interrupts are used for tasks that do not require immediate hardware response.

For example:



This keeps the hardware interrupt handler short.

Example

Suppose a high-speed sensor generates an interrupt.

The ISR:

1. Identify sensor
2. Read status
3. Store data/buffer address
4. Clear interrupt
5. Trigger deferred processing
6. Return

The actual data analysis is performed later.

Result

The proposed mechanism provides:

- Nested interrupt support
- Fast response
- <10 μ s target latency
- Efficient high-priority device handling
- Reduced ISR execution time
- Background processing through software interrupts